



Advanced VLSI Design Exercises

Day 7: Bus and DMA controller

Hiroki Matsutani

matutani@arc.ics.keio.ac.jp

Class schedule (as of May 21)

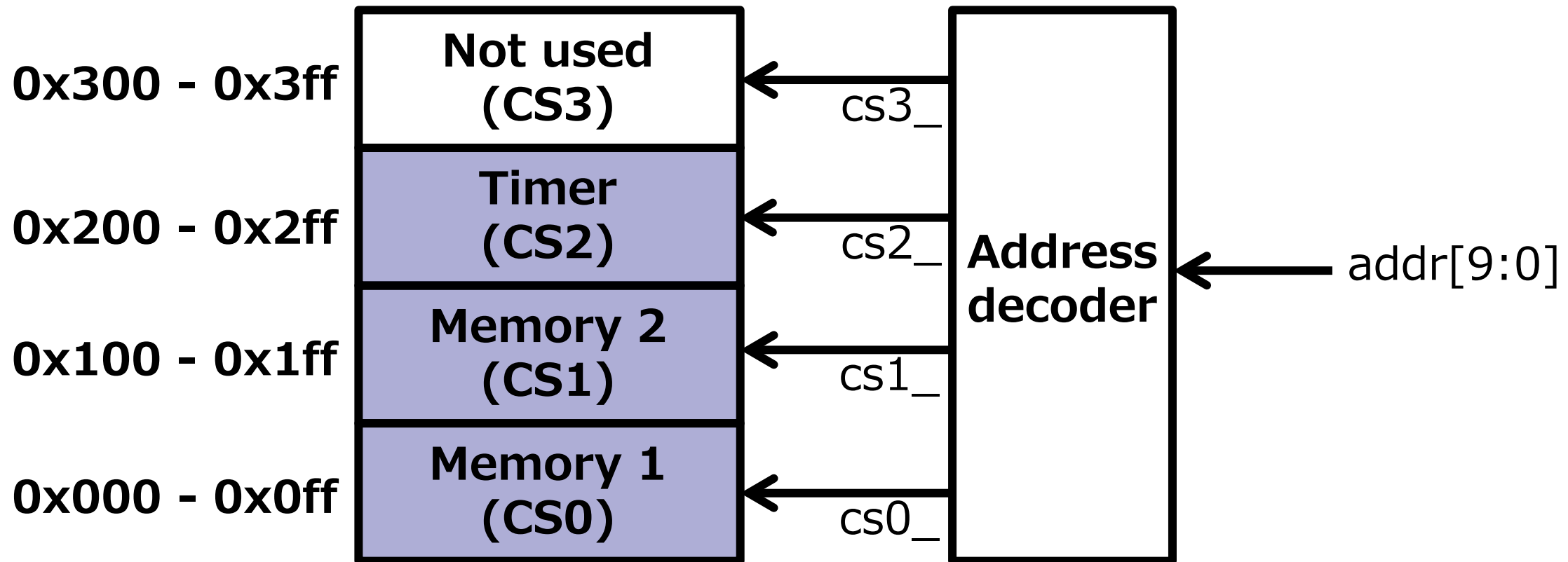
- **Day 1 (Apr 9)**
Introduction
- **Day 2 (Apr 16)**
Standard cell based design
- **Day 3 (Apr 23)**
Verilog HDL: Basics
- **Day 4 (Apr 30)**
Verilog HDL: Exercises
- **Day 5 (May 7)**
Processor architecture
- **Day 6 (May 14)**
Memory map and I/O devices
- **Day 7 (May 21)**
Bus and DMA controller
- **Day 8 (May 28)**
Logic synthesis
- **Day 9 (Jun 11)**
Placement and routing
- **Day 10 (Jun 18)**
Delay and power analysis
- **Day 11 (Jun 25)**
Design exercises
- **Day 12 (Jul 2)**
Hierarchical design
- **Day 13 (Jul 9)**
Chip assembly
- **Day 14 (Jul 16)**
Final report

Schedule may be changed. We will let you know when it is changed. 2

Day 6: Memory map and I/O devices

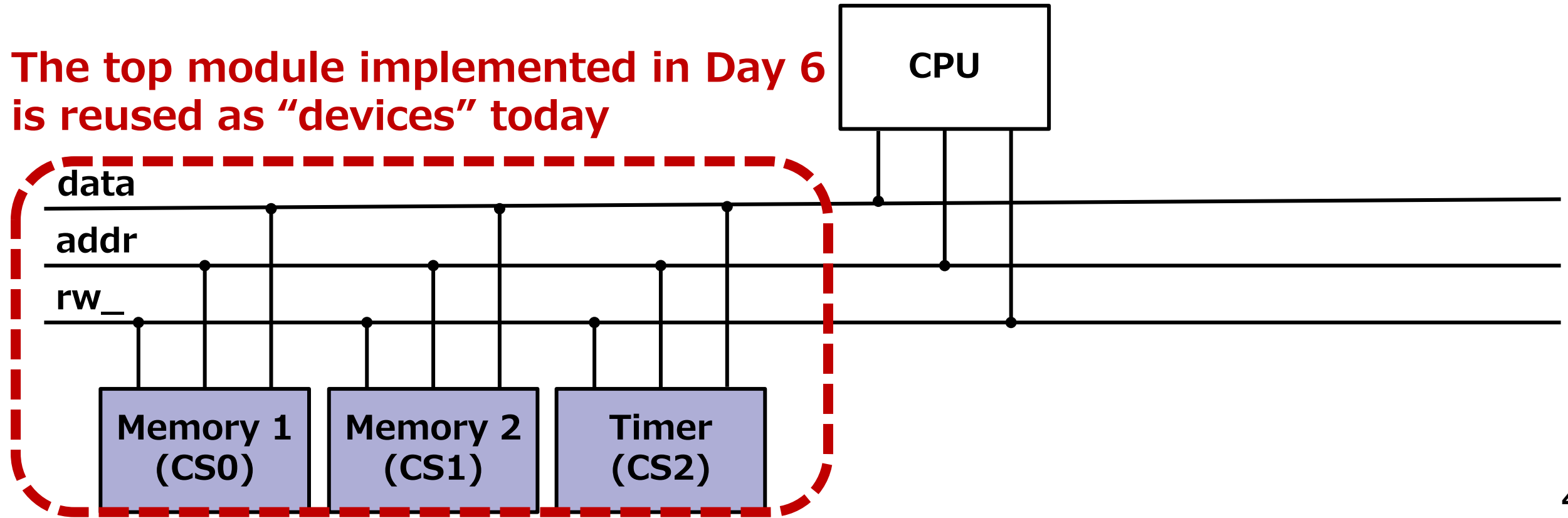
- Address decoder
- Memory and I/O devices

CS0: Memory 1, CS1: Memory 2, CS2: Timer, CS3: Not used



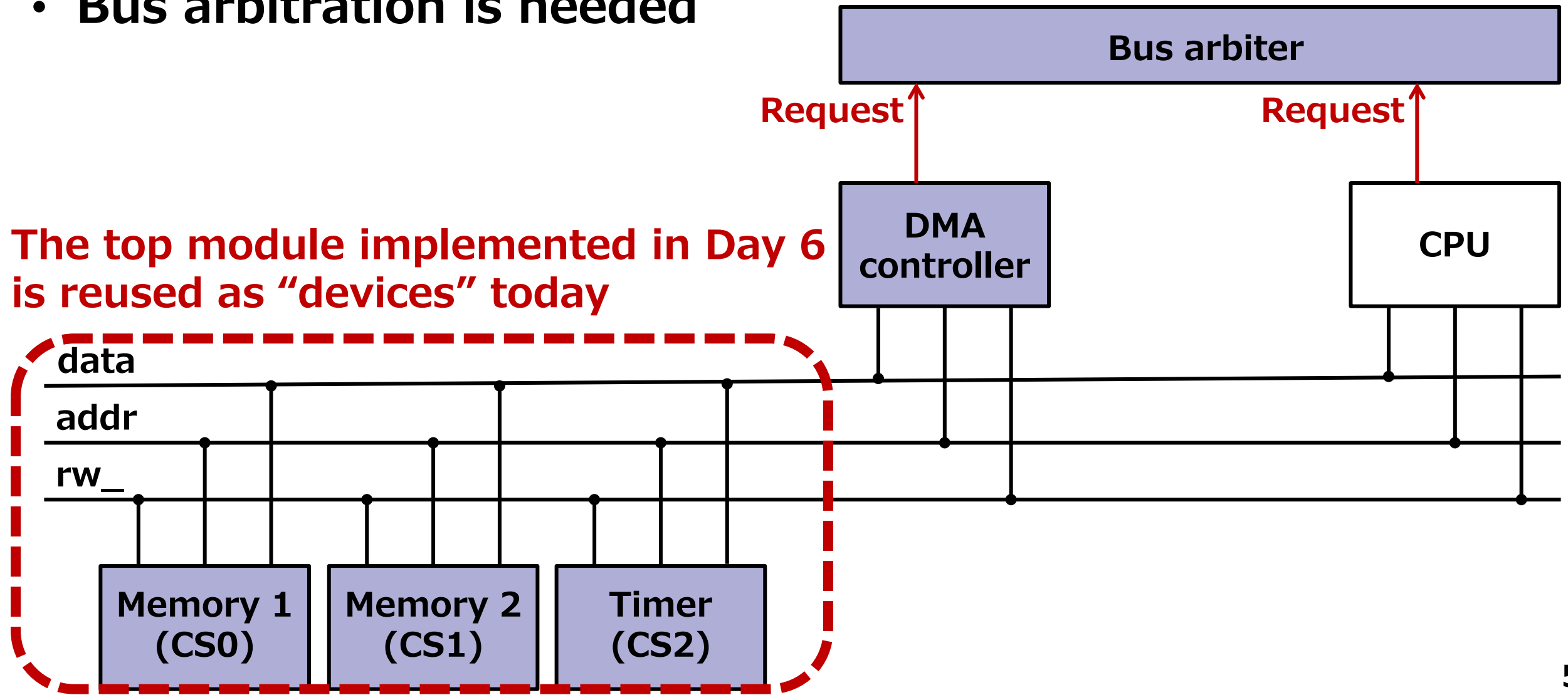
Day 6: Memory map and I/O devices

- The three devices are connected by an on-chip bus
- We assumed an external CPU is an initiator device



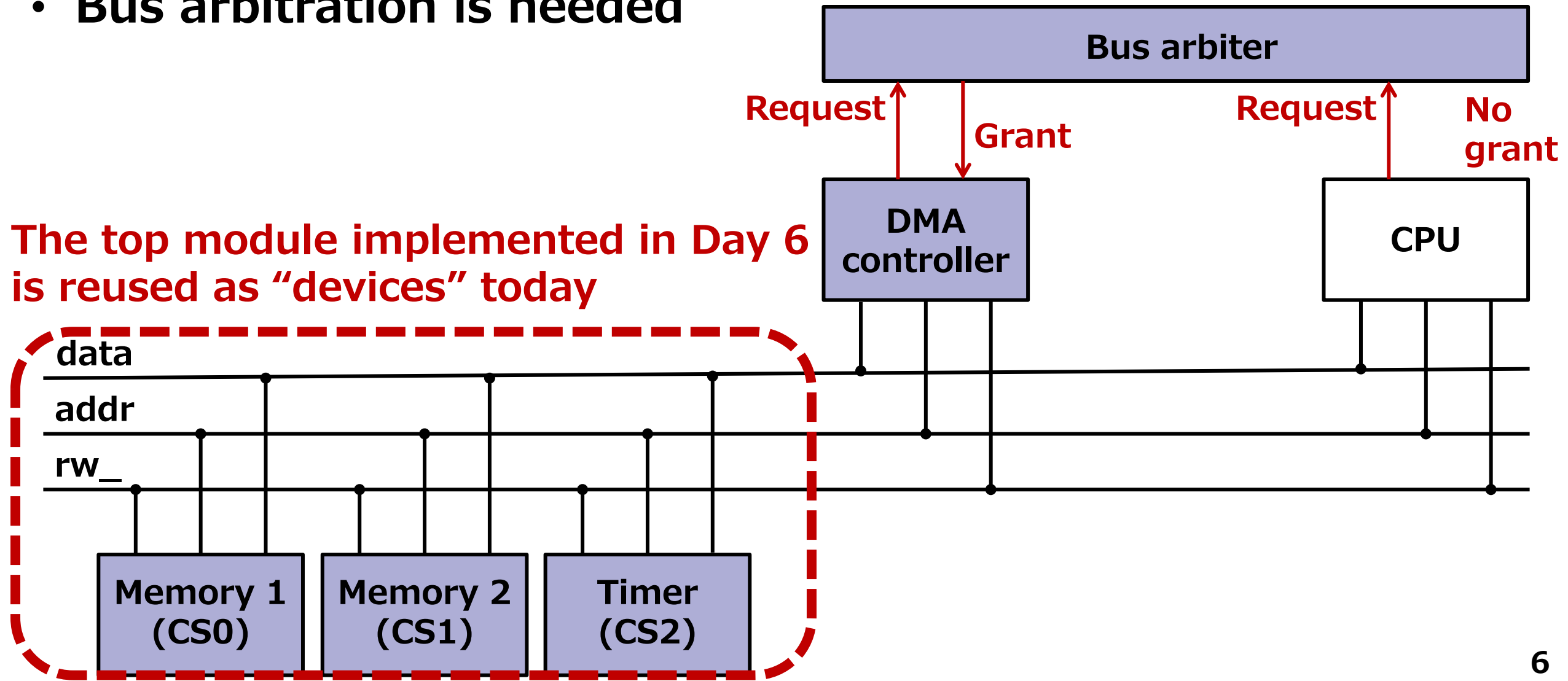
Day 7: Bus and DMA controller

- Multiple initiator devices: CPU and DMA controller
- Bus arbitration is needed



Day 7: Bus and DMA controller

- Multiple initiator devices: CPU and DMA controller
- Bus arbitration is needed



Ex1: Implementation of bus arbiter

- **Copy today's materials**
 - > `cp ~matutani/files_7.zip .`
 - > `unzip files_7.zip`
- **Incomplete source files**
 - `define.h`: Header file
 - `busarb.v`: Bus arbiter
 - `devices.v`: Day 6's top module
 - `dmactr.v`: DMA controller
 - `top.v`: Top module
 - `test.v`: Test bench

Ex1: Implementation of bus arbiter

- **Signals**
 - Bus request from initiator 1 (breq0_)
 - Bus grant to initiator 1 (bgrt0_)
 - Bus request from initiator 2 (breq1_)
 - Bus grant to initiator 2 (bgrt1_)
- **Specification**
 - Once a bus access is granted to an initiator device, the grant is kept until the initiator de-asserts its request

Ex1: Implementation of bus arbiter

```
`include "define.h"
module busarb (
    // Bus request/grant for initiators 1 and 2
    input breq0_, input breq1_, output bgrt0_, output bgrt1_,
    input reset_, input clk);

    reg state;
    assign bgrt0_ = (state == `BGRT0) ? `Enable_ : `Disable_;
    assign bgrt1_ = ...
```

```
/* define.h */
`define BGRT0      0
`define BGRT1      1
...

```

Ex1: Implementation of bus arbiter

```
`include "define.h"
module busarb (
    // Bus request/grant for initiators 1 and 2
    input breq0_, input breq1_, output bgrt0_, output bgrt1_,
    input reset_, input clk);

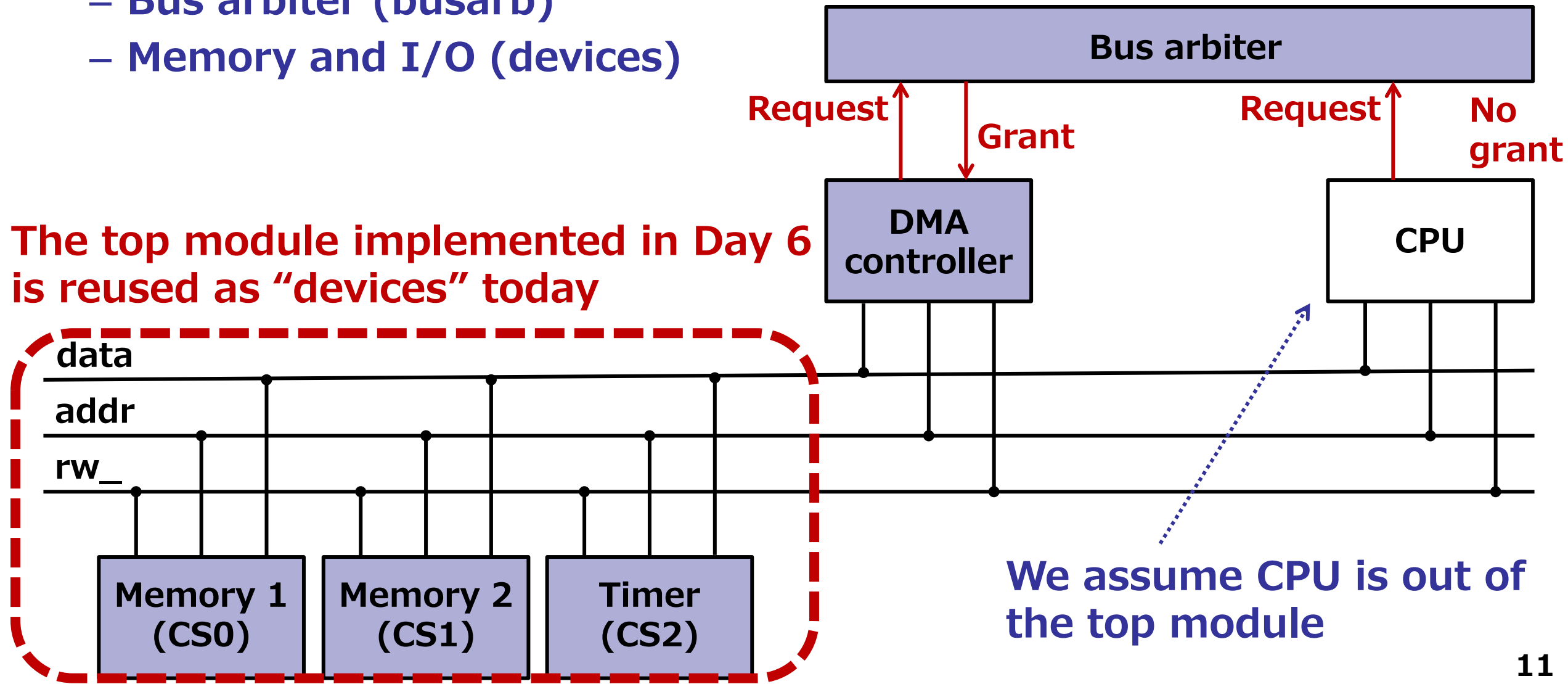
    reg state;
    assign bgrt0_ = (state == `BGRT0) ? `Enable_ : `Disable_;
    assign bgrt1_ = ...
    always @ (posedge clk)
        if (reset_ == `Enable_) begin
            // State register should be initialized
        end else begin
            case (state)
                `BGRT0: if (breq0_==`Disable_ && breq1_==`Enable_)
                        state <= `BGRT1;
                `BGRT1: if (breq1_==`Disable_ && breq0_==`Enable_)
                        state <= `BGRT0;
            endcase
        end
```

```
/* define.h */
`define BGRT0      0
`define BGRT1      1
...
```

Ex2: Implementation of top module

- Top module includes
 - Bus arbiter (busarb)
 - Memory and I/O (devices)

The top module implemented in Day 6 is reused as “devices” today



Ex2: Implementation of top module

```
module top (  
    // Initiator 1  
    input  [`BUS_ADDR_WIDTH-1:0] addr0,  
    input  [`DATA_WIDTH-1:0]      idata0,  
    output [`DATA_WIDTH-1:0]      odata0,  
    input  rw0_, input breq0_, output bgrt0_,  
    // Initiator 2  
    input  [`BUS_ADDR_WIDTH-1:0] addr1,  
    input  [`DATA_WIDTH-1:0]      idata1,  
    output [`DATA_WIDTH-1:0]      odata1,  
    input  rw1_, input breq1_, output bgrt1_,  
    input  reset_, input clk);
```

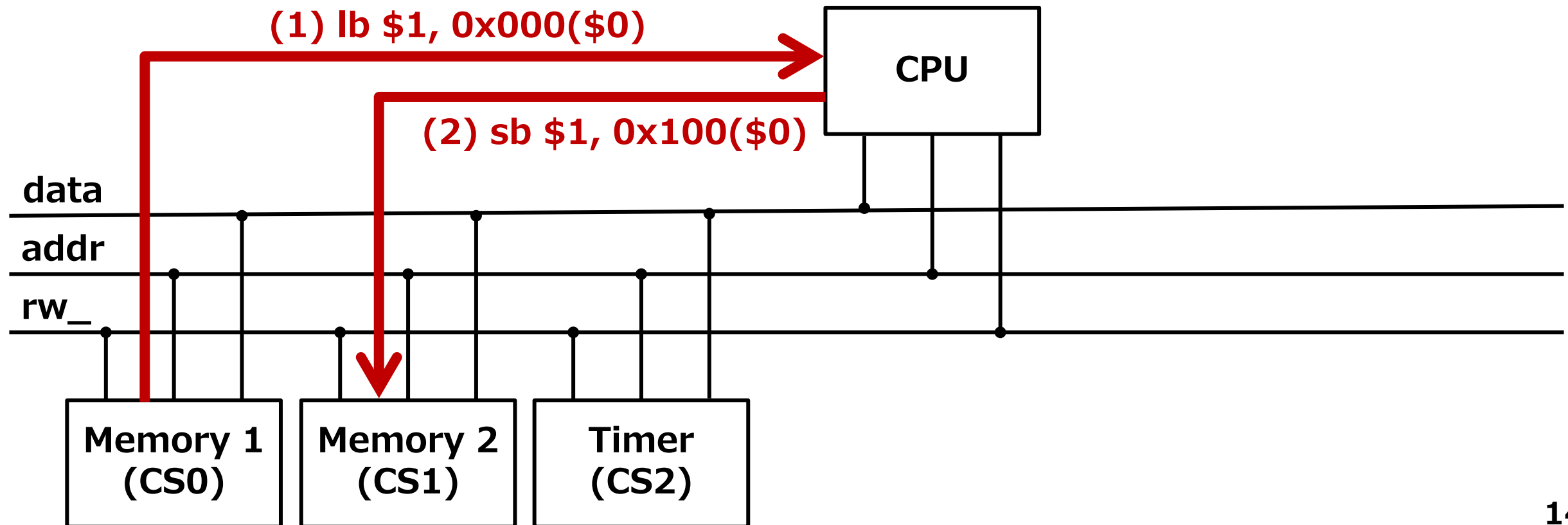
```
devices u0 (addr, idata, odata, rw_, reset_, clk);  
busarb u1 (breq0_, breq1_, bgrt0_, bgrt1_, reset_, clk);
```

Ex2: Implementation of top module

```
module top (  
    // Initiator 1  
    input  [`BUS_ADDR_WIDTH-1:0] addr0,  
    input  [`DATA_WIDTH-1:0]      idata0,  
    output [`DATA_WIDTH-1:0]      odata0,  
    input  rw0_, input breq0_, output bgrt0_,  
    // Initiator 2  
    <<Snip>>  
    input  reset_, input clk);  
  
    wire [`BUS_ADDR_WIDTH-1:0] addr;  
    wire [`DATA_WIDTH-1:0] idata, odata;  
    wire rw_  
    assign odata0 = odata;  
    assign odata1 = odata;  
    assign addr  = // addr0 or addr1 depending on bgrt  
    assign idata = // idata0 or idata1 depending on bgrt  
    assign rw_   = // rw0_ or rw1_ depending on bgrt  
    devices u0 (addr, idata, odata, rw_, reset_, clk);  
    busarb u1 (breq0_, breq1_, bgrt0_, bgrt1_, reset_, clk);
```

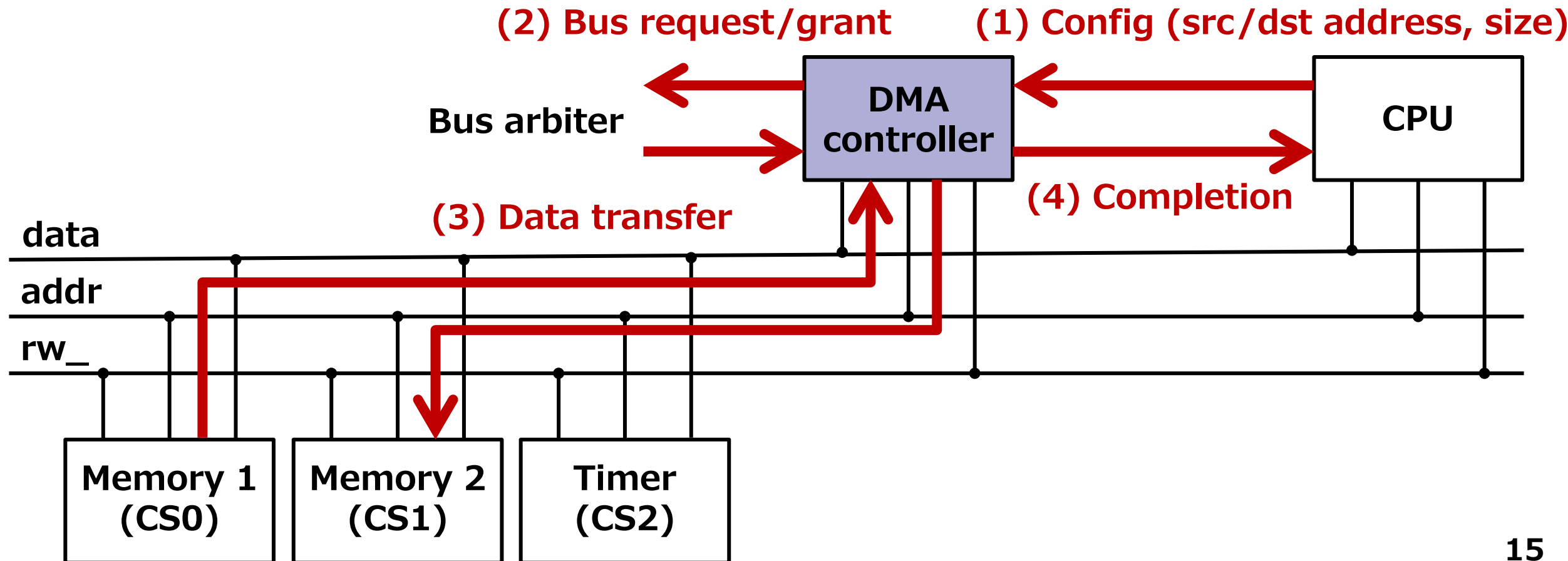
Data transfer: Programmed I/O

- Data transfer by CPU using load and store instructions
- E.g., 64-word transfer from memory 1 to memory 2
 - CPU loads a word from memory 1 and stores it to memory 2
 - Repeats 64 times



Data transfer: Direct Memory Access

- DMA transfer is configured by CPU
- Once DMA request is asserted, DMAC works as follows:
 - Bus grant, data transfer, bus release, and completion notification



Ex3: Adding an empty DMA controller

- DMA controller logic is not implemented yet

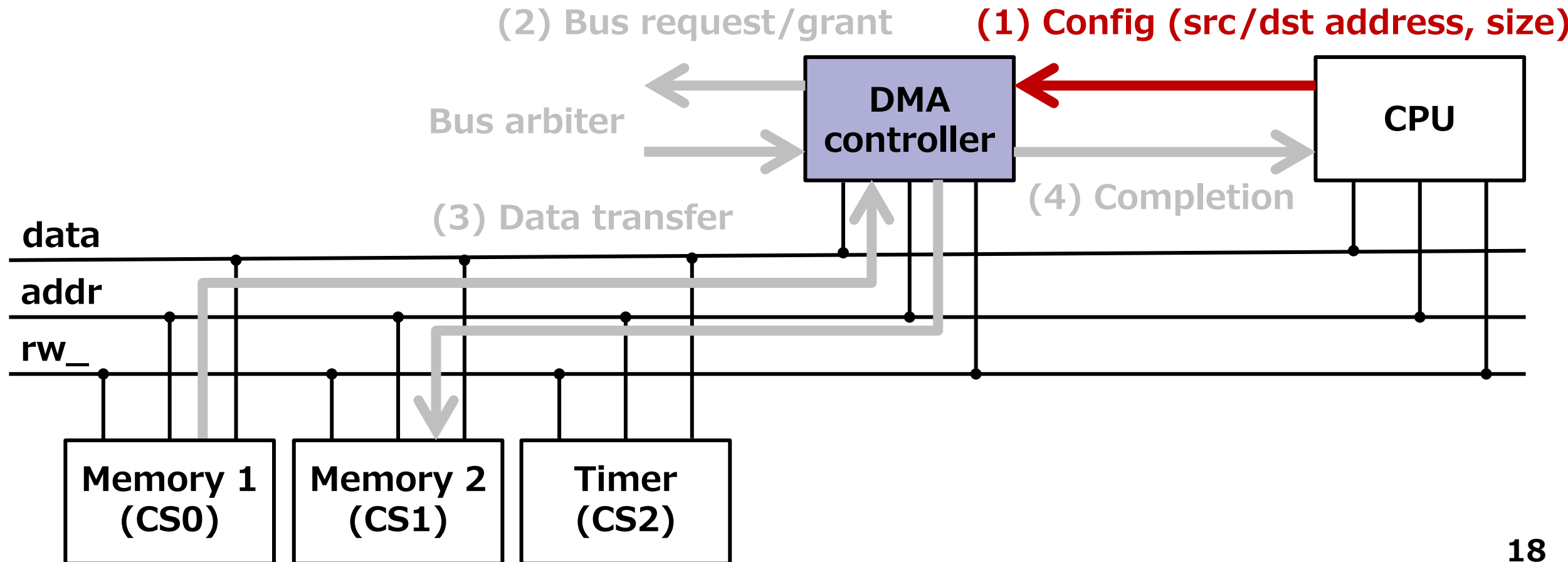
```
module dmactr (  
    // Signals between DMAC and devices  
    output reg [`BUS_ADDR_WIDTH-1:0] addr, // Memory or I/O address  
    output reg [`DATA_WIDTH-1:0] odata, // Write data  
    input [`DATA_WIDTH-1:0] idata, // Read data  
    output reg rw_, // Read/write signal  
    // Signals between DMAC and bus arbiter  
    output reg breq_, // Bus request  
    input bgrt_, // Bus grant  
    input reset_,  
    input clk);  
  
    // Not implemented yet  
  
endmodule
```

Ex3: Adding an empty DMA controller

```
module top (  
    // Initiator 1 (external CPU)  
    input  [`BUS_ADDR_WIDTH-1:0] addr0,  
    input  [`DATA_WIDTH-1:0]      idata0,  
    output [`DATA_WIDTH-1:0]      odata0,  
    input  rw0_, input breq0_, output bgrt0_,  
    input  reset_, input clk);  
    // Initiator 2 (DMAC)  
    wire  [`BUS_ADDR_WIDTH-1:0] addr1;  
    wire  [`DATA_WIDTH-1:0] idata1, odata1;  
    wire  rw1_, breq1_, bgrt1_;  
  
    <<Snip>>  
  
    assign addr  = // addr0 or addr1 depending on bgrt  
    assign idata = // idata0 or idata1 depending on bgrt  
    assign rw_   = // rw0_ or rw1_ depending on bgrt  
    devices u0 (addr, idata, odata, rw_, reset_, clk);  
    busarb u1 (breq0_, breq1_, bgrt0_, bgrt1_, reset_, clk);  
    dmactr u2 (addr1, idata1, odata1, rw1_, breq1_, bgrt1_, reset_, clk);
```

Ex4: DMA configuration by CPU

- Source address, destination address, and transfer size
 - Single transfer (1 word), burst transfer (e.g., 4 words)
 - Transfer type: Memory-to-memory, Memory-to-I/O, I/O-to-memory



Ex4: DMA configuration by CPU

- **Transfer mode**
 - Single: Bus grant → 1-word transfer → Bus release
 - Burst: Bus grant → n-word transfer → Bus release (e.g., n=4)
- **Transfer type (burst mode only)**
 - Memory-to-memory: Both **src**/**dst** addresses are incremented at every single transfer
 - Memory-to-I/O: Only **src** address is incremented at every transfer
 - I/O-to-memory: Only **dst** address is incremented at every transfer

Ex4: DMA configuration by CPU

- **Transfer mode**
 - Single: Bus grant → 1-word transfer → Bus release
 - Burst: Bus grant → n-word transfer → Bus release (e.g., n=4)
- **Transfer type (burst mode only)**
 - Memory-to-memory: Both **src**/**dst** addresses are incremented at every single transfer
 - Memory-to-I/O: Only **src** address is incremented at every transfer
 - I/O-to-memory: Only **dst** address is incremented at every transfer
- **Single transfer is designed during class (you might want to design burst mode in your short report)**
 - Mode 00: Single
 - Mode 01: Burst (memory-to-memory)
 - Mode 10: Burst (memory-to-I/O)
 - Mode 11: Burst (I/O-to-memory)

Ex4: DMA configuration by CPU

- DMA controller logic is not implemented yet

```
module dmactr (  
    // Signals between DMAC and devices  
    output reg [`BUS_ADDR_WIDTH-1:0] addr, // Memory or I/O address  
    output reg [`DATA_WIDTH-1:0] odata, // Write data  
    input [`DATA_WIDTH-1:0] idata, // Read data  
    output reg rw_, // Read/write signal  
    // Signals between DMAC and bus arbiter  
    output reg breq_, // Bus request  
    input bgrt_, // Bus grant  
    // Signals between DMAC and CPU (DMA configuration)  
    input [`BUS_ADDR_WIDTH-1:0] dsaddr, // DMA source address  
    input [`BUS_ADDR_WIDTH-1:0] ddaddr, // DMA destination address  
    input [1:0] dmode, // DMA transfer mode  
    input reset_,  
    input clk);  
  
    // Not implemented yet
```

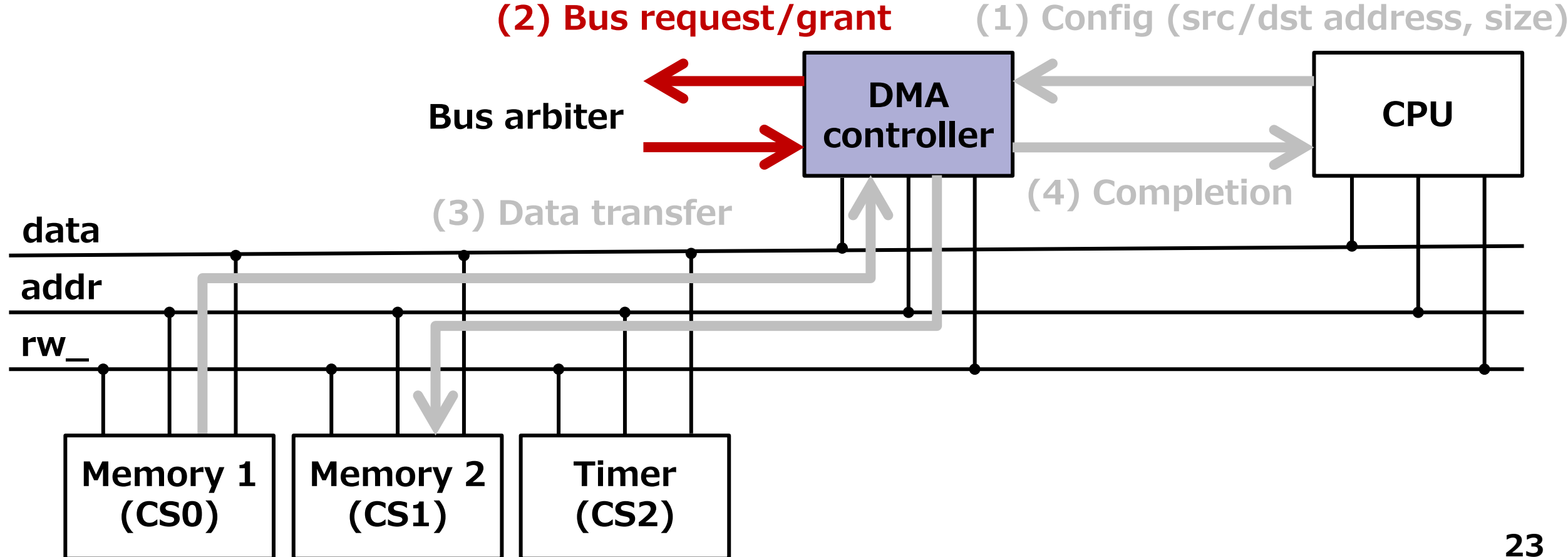
Ex4: DMA configuration by CPU

```
module top (  
    // Initiator 1 (external CPU)  
    input  [`BUS_ADDR_WIDTH-1:0] addr0,  
    input  [`DATA_WIDTH-1:0]      idata0,  
    output [`DATA_WIDTH-1:0]      odata0,  
    input  rw0_, input breq0_, output bgrt0_,  
    // Signals between DMAC and CPU (DMA configuration)  
    input  [`BUS_ADDR_WIDTH-1:0] dsaddr, // DMA source address  
    input  [`BUS_ADDR_WIDTH-1:0] ddaddr, // DMA destination address  
    input  [1:0]                  dmode,  // DMA transfer mode  
    input  reset_, input clk);  
  
    <<Snip>>  
  
    devices u0 (addr, idata, odata, rw_, reset_, clk);  
    busarb u1 (breq0_, breq1_, bgrt0_, bgrt1_, reset_, clk);  
    dmactr u2 (addr1, idata1, odata1, rw1_, breq1_, bgrt1_,  
              dsaddr, ddaddr, dmode, reset_, clk);
```

Ex5: DMA request and bus grant

- DMA controller

- Once DMA request (dreq) is received, assert bus request (breq)
- Once bus grant (bgrt) is received, start the DMA transfer (Note breq should be kept until the transfer completes)



Ex5: DMA request and bus grant

```
module dmactr (  
    // Signals between DMAC and devices  
    output reg [`BUS_ADDR_WIDTH-1:0] addr, // Memory or I/O address  
    output reg [`DATA_WIDTH-1:0] odata, // Write data  
    input [`DATA_WIDTH-1:0] idata, // Read data  
    output reg rw_, // Read/write signal  
    // Signals between DMAC and bus arbiter  
    output reg breq_, // Bus request  
    input bgrt_, // Bus grant  
    // Signals between DMAC and CPU  
    input [`BUS_ADDR_WIDTH-1:0] dsaddr, // DMA source address  
    input [`BUS_ADDR_WIDTH-1:0] ddaddr, // DMA destination address  
    input [1:0] dmode, // DMA transfer mode  
    input dreq_, // DMA request (*)  
    input reset_,  
    input clk);  
    // Not implemented yet  
    (* We assume DMA request is  
    triggered by CPU in this exercise  
  
endmodule
```

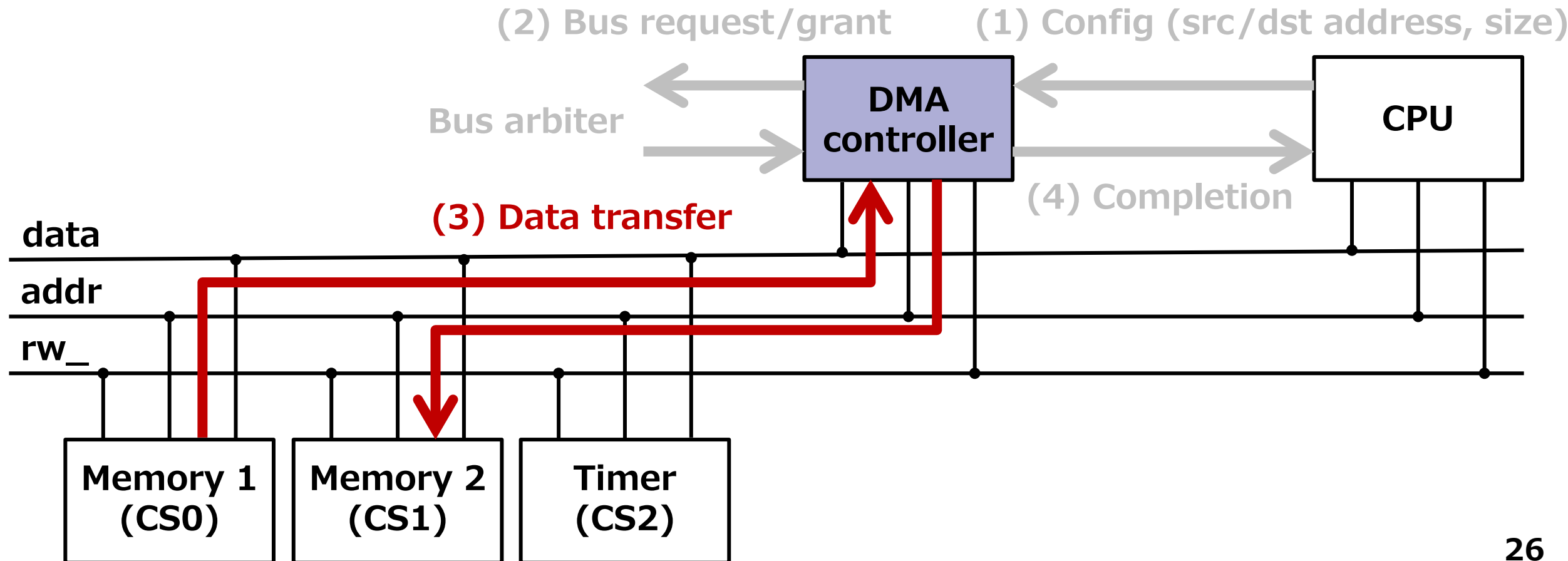
Ex5: DMA request and bus grant

```
module top (  
    // Initiator 1 (external CPU)  
    input  [`BUS_ADDR_WIDTH-1:0] addr0,  
    input  [`DATA_WIDTH-1:0]      idata0,  
    output [`DATA_WIDTH-1:0]      odata0,  
    input  rw0_, input breq0_, output bgrt0_,  
    // Signals between DMAC and CPU  
    input  [`BUS_ADDR_WIDTH-1:0] dsaddr, // DMA source address  
    input  [`BUS_ADDR_WIDTH-1:0] ddaddr, // DMA destination address  
    input  [1:0]                  dmode,  // DMA transfer mode  
    input                          dreq_,  // DMA request (*)  
    input  reset_, input clk);  
  
    <<Snip>>  
  
    devices u0 (addr, idata, odata, rw_, reset_, clk);  
    busarb u1 (breq0_, breq1_, bgrt0_, bgrt1_, reset_, clk);  
    dmactr u2 (addr1, idata1, odata1, rw1_, breq1_, bgrt1_,  
              dsaddr, ddaddr, dmode, dreq_, reset_, clk);
```

(*) We assume DMA request is triggered by CPU in this exercise

Ex6: Data transfer by DMAC

- DMA controller
 - Read 1-word from src address and store it to dst address
 - Use `addr`, `odata`, `idata`, and `rw_` for the data transfer



Ex6: Data transfer by DMAC

```
module dmactr (  
    // Signals between DMAC and devices  
    output reg [`BUS_ADDR_WIDTH-1:0] addr, // Memory or I/O address  
    output reg [`DATA_WIDTH-1:0] odata, // Write data  
    input [`DATA_WIDTH-1:0] idata, // Read data  
    output reg rw_, // Read/write signal  
    // Signals between DMAC and bus arbiter  
    output reg breq_, // Bus request  
    input bgrt_, // Bus grant  
    // Signals between DMAC and CPU  
    input [`BUS_ADDR_WIDTH-1:0] dsaddr, // DMA source address  
    input [`BUS_ADDR_WIDTH-1:0] ddaddr, // DMA destination address  
    input [1:0] dmode, // DMA transfer mode  
    input dreq_, // DMA request  
    input reset_,  
    input clk);  
    // Once dreq_ is asserted, assert breq_ and wait until bgrt_ is asserted
```

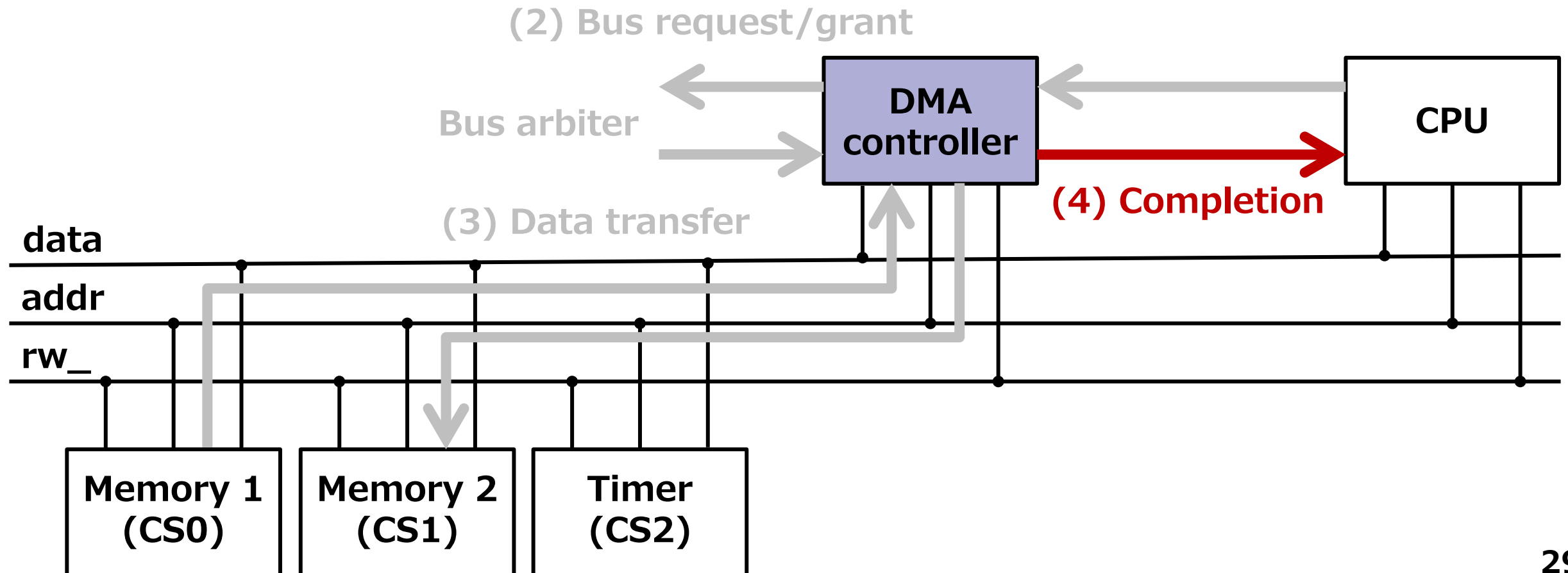
```
endmodule
```

Ex6: Data transfer by DMAC

```
module dmactr (  
    // Signals between DMAC and devices  
    output reg [`BUS_ADDR_WIDTH-1:0] addr, // Memory or I/O address  
    output reg [`DATA_WIDTH-1:0] odata, // Write data  
    input [`DATA_WIDTH-1:0] idata, // Read data  
    output reg rw_, // Read/write signal  
    // Signals between DMAC and bus arbiter  
    output reg breq_, // Bus request  
    input bgrt_, // Bus grant  
    // Signals between DMAC and CPU  
    input [`BUS_ADDR_WIDTH-1:0] dsaddr, // DMA source address  
    input [`BUS_ADDR_WIDTH-1:0] ddaddr, // DMA destination address  
    input [1:0] dmode, // DMA transfer mode  
    input dreq_, // DMA request  
    input reset_,  
    input clk);  
    // Once dreq_ is asserted, assert breq_ and wait until bgrt_ is asserted  
    // Once bgrt_ is asserted, start DMA transfer  
    // (Read a single word from dsaddr and write it to ddaddr)  
endmodule
```

Ex7: DMA transfer completion

- DMA controller
 - Notify completion of the DMA transfer to the CPU
 - Assert eop_ after the transfer completes



Ex7: DMA transfer completion

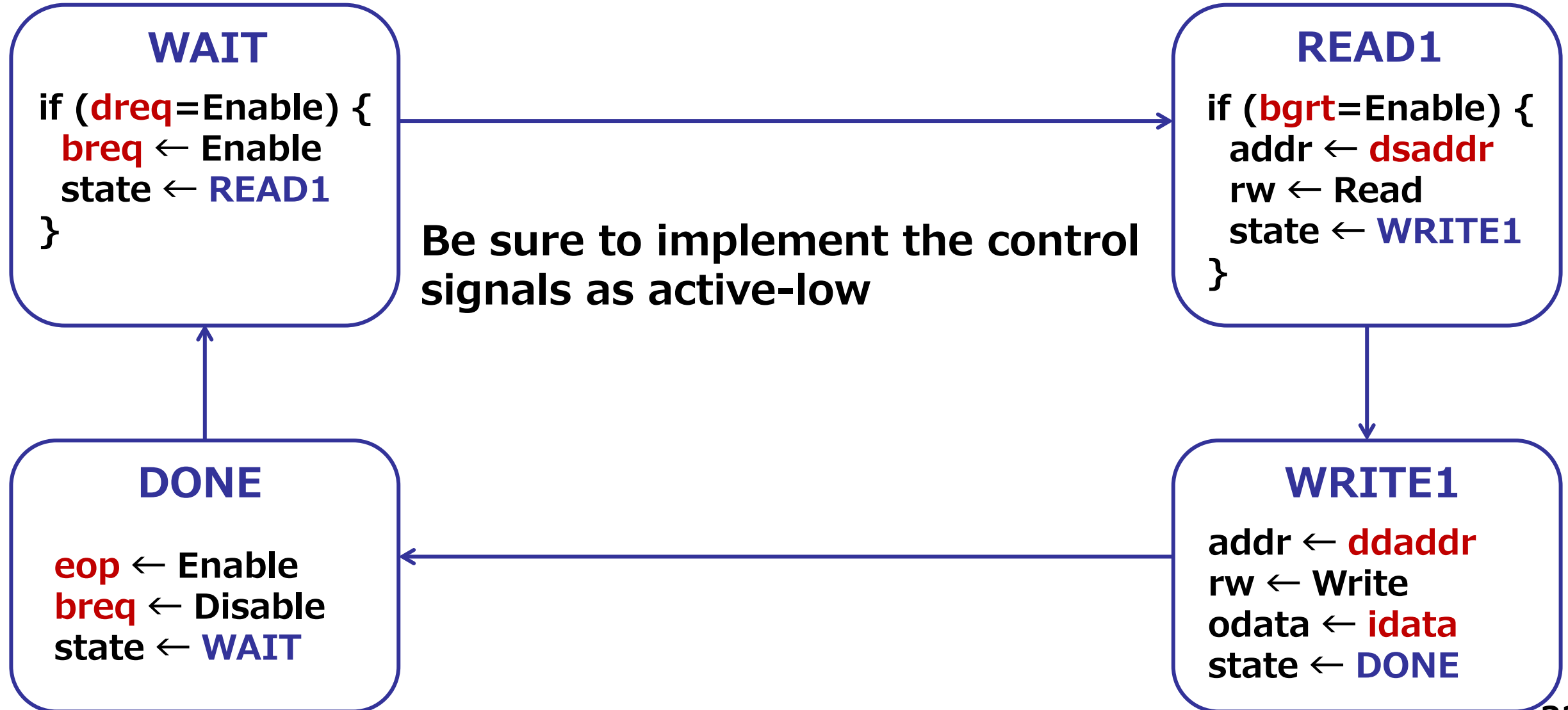
```
module dmactr (  
    // Signals between DMAC and devices  
    output reg [`BUS_ADDR_WIDTH-1:0] addr, // Memory or I/O address  
    output reg [`DATA_WIDTH-1:0] odata, // Write data  
    input [`DATA_WIDTH-1:0] idata, // Read data  
    output reg rw_, // Read/write signal  
    // Signals between DMAC and bus arbiter  
    output reg breq_, // Bus request  
    input bgrt_, // Bus grant  
    // Signals between DMAC and CPU  
    input [`BUS_ADDR_WIDTH-1:0] dsaddr, // DMA source address  
    input [`BUS_ADDR_WIDTH-1:0] ddaddr, // DMA destination address  
    input [1:0] dmode, // DMA transfer mode  
    input dreq_, // DMA request  
    output reg eop_, // DMA complete  
    input reset_,  
    input clk);  
  
    // Once DMA transfer completes, assert eop_ and de-assert breq_  
endmodule
```

Ex7: DMA transfer completion

```
module top (  
    // Initiator 1 (external CPU)  
    input  [`BUS_ADDR_WIDTH-1:0] addr0,  
    input  [`DATA_WIDTH-1:0]      idata0,  
    output [`DATA_WIDTH-1:0]      odata0,  
    input  rw0_, input breq0_, output bgrt0_,  
    // Signals between DMAC and CPU  
    input  [`BUS_ADDR_WIDTH-1:0] dsaddr, // DMA source address  
    input  [`BUS_ADDR_WIDTH-1:0] ddaddr, // DMA destination address  
    input  [1:0]                  dmode,  // DMA transfer mode  
    input                          dreq_,  // DMA request  
    output                          eop_,   // DMA complete  
    input  reset_, input clk);  
  
<<Snip>>  
  
    devices u0 (addr, idata, odata, rw_, reset_, clk);  
    busarb u1 (breq0_, breq1_, bgrt0_, bgrt1_, reset_, clk);  
    dmactr u2 (addr1, idata1, odata1, rw1_, breq1_, bgrt1_,  
              dsaddr, ddaddr, dmode, dreq_, eop_, reset_, clk);
```

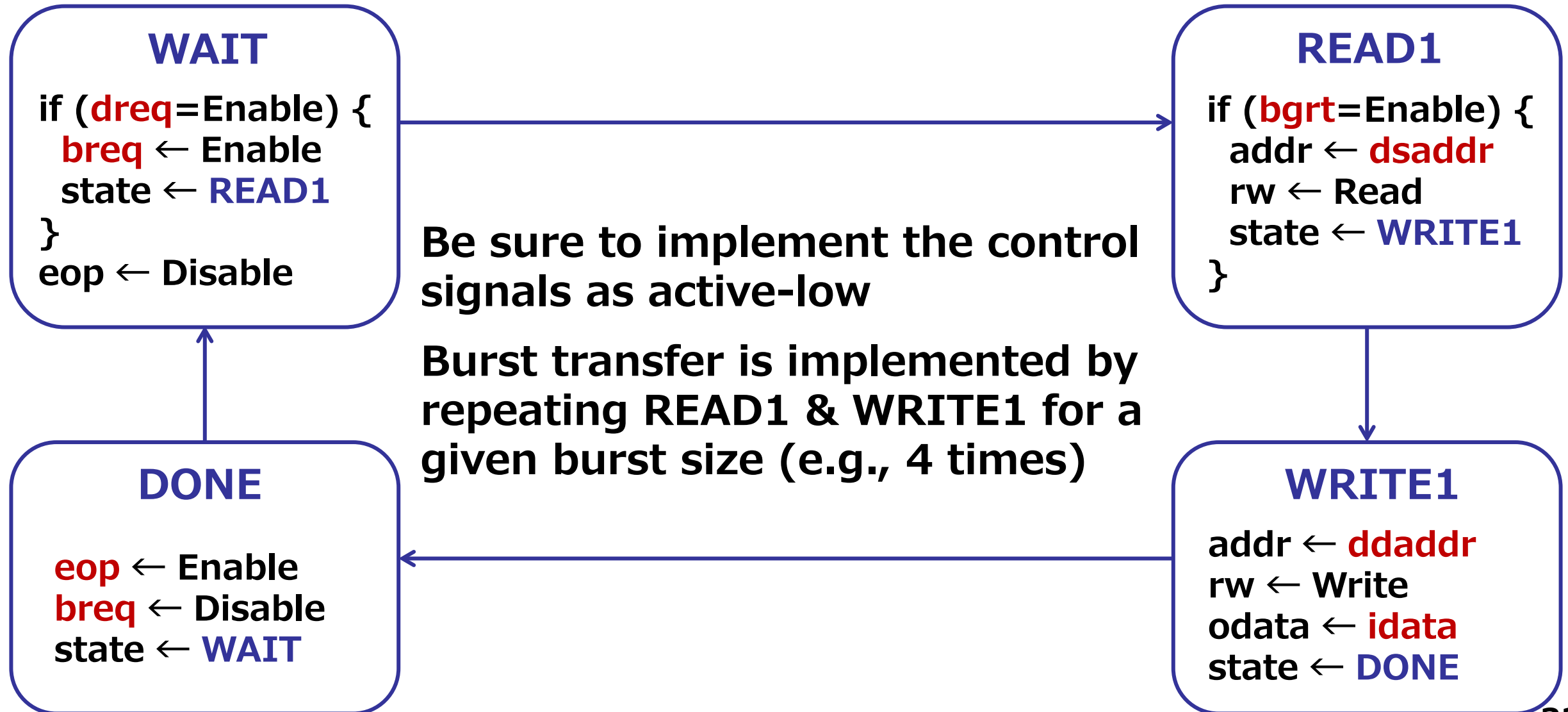
Ex6 & Ex7: State transition diagram

- A state transition diagram for Ex6 & Ex7



Ex6 & Ex7: State transition diagram

- A state transition diagram for Ex6 & Ex7



Ex6 & Ex7: State transition diagram

- An example state machine code for Ex6 & Ex7

```
always @ (posedge clk)
    if (reset_ == `Enable_) begin
        // Registers should be initialized
    end else begin
        case (state)
            `WAIT: begin
                ...
            end
            `READ1: begin
                ...
            end
            `WRITE1: begin
                ...
            end
            `DONE: begin
                ...
            end
        endcase
    end
```

```
/* define.h */
`define WAIT    2'b00
`define READ1   2'b01
`define WRITE1  2'b10
`define DONE    2'b11
...
```

Ex6 & Ex7: Test procedure (1/3)

1. Reset

- Assert reset_ for several cycles and then de-assert it

2. Memory write by CPU (initiator 1)

- Write a value of 0x99 to memory address of 0x150
- Control addr0, idata0, rw0_, and breq0_ of the top module

3. Memory read by CPU (initiator 1)

- Read a value from memory address of 0x150 and display the value
- Use \$display() to display the value (e.g., odata0)

Ex6 & Ex7: Test procedure (2/3)

4. Data transfer by DMAC (initiator 2)

- Single transfer from 0x150 to 0x160
- Control dsaddr, ddaddr, dmode, and dreq_
- Note breq0_ should be de-asserted beforehand; otherwise, DMAC cannot access the bus
- Note dreq_ should be de-asserted once transfer gets started so as not to execute the transfer again

5. Memory read by CPU (initiator 1)

- Display the value of memory address of 0x160
- Be careful of breq0_ and breq1_

Ex6 & Ex7: Test procedure (3/3)

```
module test ();
reg [`BUS_ADDR_WIDTH-1:0] addr0;
reg [`DATA_WIDTH-1:0] idata0;
wire [`DATA_WIDTH-1:0] odata0;
reg rw0_, breq0_, dreq_;
wire bgrt0_, eop_;
reg [`BUS_ADDR_WIDTH-1:0] dsaddr, ddaddr;
reg [1:0] dmode;
reg reset_, clk;
top u0 (addr0, idata0, odata0, rw0_, breq0_, bgrt0_, dsaddr, ddaddr, dmode,
dreq_, eop_, reset_, clk);
initial begin
    #1; $dumpfile("dump.vcd"); $dumpvars(0, test.u0);
    // 1. Reset
    ...
    // 2. Memory write by CPU (initiator 1)
    ...
    // 3. Memory read by CPU (initiator 1)
    ...
    // 4. Data transfer by DMAC (initiator 2)
```